UNIVERSITY OF KHARTOUM

FACULTY OF ENGINEERING

DEPARTMENT OF ELECTRICAL AND ELECTRONIC ENGINEERING

MICROPROCESSOR SYSTEM DESIGN COURSE

Automatic Pet Feeding System

Submitted By : Group 34

Alshaima Abdullahi Mohmed Elbashir - 194029

Mohamed Abubaker Elsir Gafar - 204097

Supervised by

Dr. Magdi B. M. Amien

August 2026

Abstract

Automatic and timely pet feeding has become increasingly important due to modern lifestyles that often prevent pet owners from maintaining regular feeding schedules. This project presents the design, simulation, and evaluation of a microcontroller-based Automatic Pet Feeding System that provides reliable and scheduled food dispensing while continuously monitoring the food level inside the storage container.

The proposed system is built around an Arduino Uno (ATmega328P) microcontroller and integrates a DS1307 Real-Time Clock (RTC) for accurate timekeeping, an SG90 servo motor for controlling the dispensing gate, an HC-SR04 ultrasonic sensor for monitoring the remaining food level, a 16×2 I²C LCD for displaying system information, a 4×4 matrix keypad for manual feeding activation, and a buzzer together with red and green LEDs for audible and visual notifications.

The embedded software automatically dispenses food at two predefined feeding times (08:00 and 18:00) using software protection logic to prevent duplicate dispensing within the same minute. Furthermore, the system continuously monitors the food level and generates immediate visual and audible refill alerts whenever the measured distance exceeds the predefined 20 cm safety threshold. The user can also initiate manual feeding on demand using the keypad without disrupting normal system operation.

The complete hardware and software system was designed and verified using the Wokwi Online Simulator. Simulation results confirmed reliable communication among all hardware modules, accurate scheduled feeding, stable I²C communication, responsive user interaction, correct food-level monitoring, and dependable actuator operation without pin conflicts or communication errors.

The proposed system provides a practical, low-cost, and reliable embedded solution for automated pet feeding. Its modular architecture also allows future expansion through features such as wireless connectivity, mobile application integration, and remote monitoring, making it suitable for both educational and real-world embedded system applications.

Keywords: Automatic Pet Feeding System, Arduino Uno, ATmega328P, Embedded Systems, DS1307 RTC, HC-SR04 Ultrasonic Sensor, SG90 Servo Motor, I²C Communication, Wokwi Simulation.

TABLE OF CONTENT

3. Problem Statement

Maintaining a consistent feeding schedule is essential for ensuring the health, well-being, and proper growth of pets. However, the demands of modern lifestyles—including long working hours, unexpected travel, and daily responsibilities—often make it difficult for pet owners to provide meals on time. As a result, feeding schedules may become irregular, leading to delayed or even missed meals.

Irregular feeding and inconsistent portion delivery may negatively affect pets by contributing to obesity, malnutrition, digestive disorders, and behavioral problems such as stress and anxiety. Traditional feeding methods, such as leaving a large quantity of food in an open bowl throughout the day, are not an effective solution because they expose the food to contamination, reduce its freshness and quality, and provide no control over feeding schedules. Although commercial automatic pet feeders are available, many are relatively expensive or lack essential smart features, such as continuous food-level monitoring and timely refill notifications.

These challenges highlight the need for a reliable, affordable, and intelligent feeding solution. Therefore, this project proposes the design, simulation, and evaluation of a microcontroller-based Automatic Pet Feeding System capable of dispensing predefined food portions at scheduled times, continuously monitoring the remaining food level using an ultrasonic sensor, and notifying the user whenever refilling is required. By automating the feeding process, the proposed system improves feeding consistency, enhances pet welfare, and significantly reduces the need for continuous human intervention while providing greater convenience and peace of mind for pet owners.

4. Objectives

The primary objective of this project is to design, simulate, and evaluate a smart microcontroller-based Automatic Pet Feeding System that provides a reliable, accurate, and cost-effective solution for automated pet care. The proposed system is designed to dispense predefined food portions at scheduled times, continuously monitor the remaining food level, and notify the user whenever refilling is required. Through automation, the system aims to improve feeding consistency, enhance pet welfare, and significantly reduce the need for continuous human intervention.

4.1 Main Objective

To design and simulate a reliable microcontroller-based Automatic Pet Feeding System capable of dispensing predefined food portions according to predefined schedules, monitoring food availability in real time, and providing timely user notifications to ensure safe, efficient, and dependable operation.

4.2 Specific Objectives

1. Embedded Hardware Design: Design a complete embedded hardware system and develop a comprehensive electronic schematic centered on the ATmega328P microcontroller to coordinate all system functions.
2. Automatic Food Dispensing: Develop an automated food dispensing mechanism driven by an SG90 servo motor to deliver predefined and consistent food portions.
3. Real-Time Scheduling: Integrate a DS1307 Real-Time Clock (RTC) module to maintain accurate timekeeping and execute feeding operations according to predefined schedules.
4. Food-Level Monitoring and Alerts: Monitor the remaining food level using an HC-SR04 ultrasonic sensor and generate visual (Red LED) and audible (Buzzer) alerts whenever the measured distance exceeds the predefined threshold, indicating a low-food condition.
5. User Interface Development: Design a simple and user-friendly interface utilizing a 16×2 I²C LCD and a 4×4 matrix keypad to display real-time system information and enable manual feeding.
6. Software Development: Develop a structured software architecture, prepare the software flowchart, and implement reliable embedded C/C++ source code to control and coordinate all hardware components.
7. Simulation and System Validation: Perform comprehensive simulation and verification using the Wokwi Online Simulator to validate circuit operation, software functionality, and hardware–software integration before physical implementation.
8. Performance Evaluation: Evaluate the overall system performance in terms of scheduling accuracy, food-level monitoring, dispensing reliability, user interface responsiveness, and overall system stability.

5. System Requirements

The system requirements define the functional capabilities, performance criteria, and technical specifications that the proposed Automatic Pet Feeding System must satisfy to ensure reliable operation and achieve the project objectives. These requirements are classified into functional requirements, non-functional requirements, and hardware and software requirements.

5.1 Functional Requirements

1. The system shall maintain accurate real-time tracking and automatically execute scheduled feeding operations using the DS1307 Real-Time Clock (RTC).
2. The system shall automatically dispense predefined food portions at scheduled feeding times (08:00 and 18:00) without requiring human intervention.
3. The system shall allow the user to initiate manual feeding by pressing the dedicated 'A' key on the 4×4 matrix keypad.
4. The system shall continuously monitor the remaining food level inside the storage container using the HC-SR04 ultrasonic sensor.
5. The system shall generate immediate visual (Red LED) and audible (Buzzer) alerts whenever the measured distance exceeds the predefined threshold of 20 cm, indicating a low-food condition.
6. The system shall display essential information, including the current time, food status, and system messages, on the 16×2 LCD with an I²C interface.

5.2 Non-Functional Requirements

1. Reliability: The system shall operate continuously and reliably while maintaining accurate feeding schedules without unexpected failures.
2. Accuracy: The system shall provide precise timekeeping and consistent operation during every feeding cycle.
3. Usability: The user interface shall be simple, intuitive, and easy to operate for users with different technical backgrounds.
4. Efficiency: The system shall utilize hardware resources efficiently while maintaining continuous and stable system operation.
5. Cost-Effectiveness: The system shall utilize affordable and widely available electronic components without compromising reliability or maintainability.

5.3 Hardware and Software Requirements

5.3.1 Hardware Requirements

- Microcontroller: Arduino Uno R3 (ATmega328P) for overall system control and processing.
- Real-Time Clock: DS1307 RTC module for accurate timekeeping through the I²C interface.
- Actuator: SG90 Micro Servo Motor for operating the food dispensing gate.
- Food-Level Sensor: HC-SR04 Ultrasonic Sensor for non-contact distance measurement and food-level monitoring.
- Display: 16×2 LCD with I²C (PCF8574) interface for displaying the current time, food status, and system messages.

- User Input: 4×4 Matrix Keypad for manual feeding commands.
- Alert Indicators: Active Piezo Buzzer, Red LED (low-food warning), and Green LED (power indicator).
- Power Supply: Regulated 5 V DC external power supply.

5.3.2 Software Requirements

- Development Environment: Arduino IDE for program development and compilation.
- Programming Language: Embedded C/C++ using the Arduino framework.
- Software Libraries:
    - Wire.h (I^2C communication)
    - LiquidCrystal_I2C.h (LCD control)
    - RTClib.h (RTC communication)
    - Servo.h (Servo motor control)
    - Keypad.h (Matrix keypad interface)
    - Simulation Platform: Wokwi Online Simulator for circuit simulation, software testing, and complete system validation.

6. Block Diagram

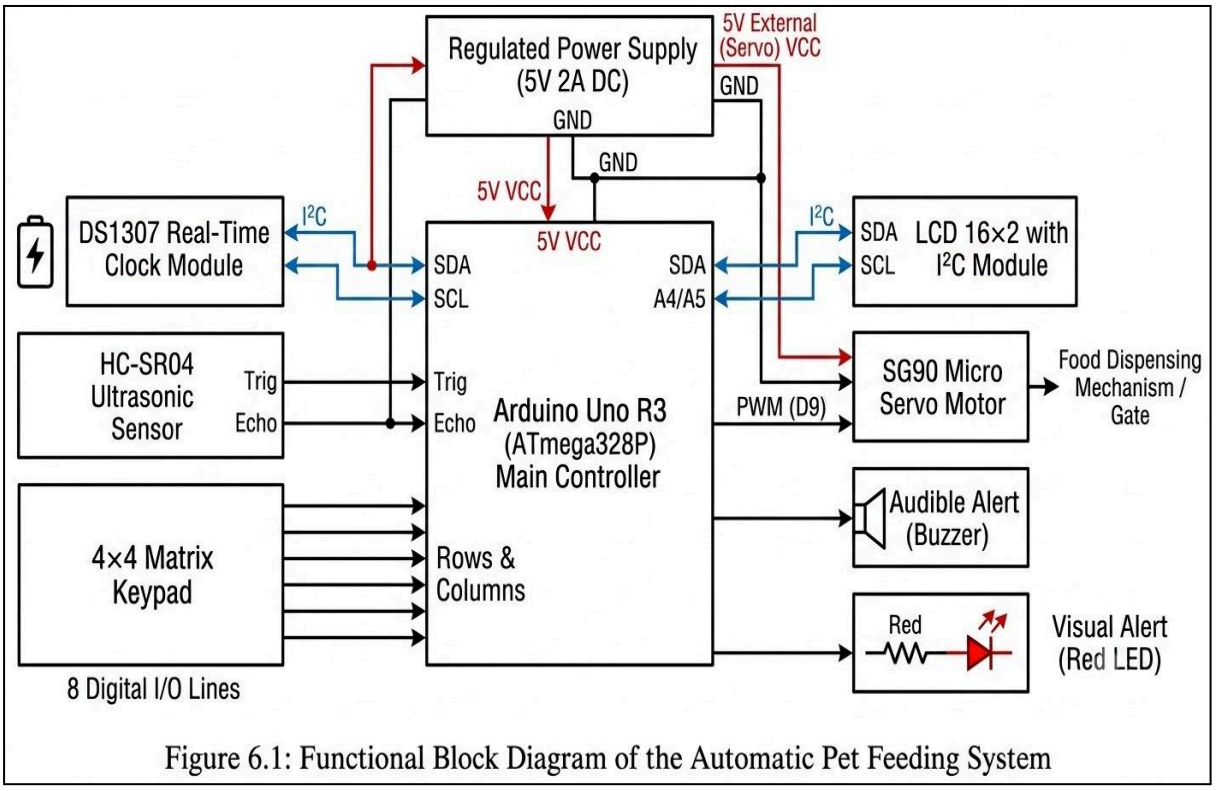

Figure 6.1: Functional Block Diagram of the Automatic Pet Feeding System

7. Hardware Architecture

This section describes the physical design and hardware organization of the Automatic Pet Feeding System. The hardware architecture follows a modular design approach, where each module performs a specific function while operating together under the supervision of a central microcontroller. This architecture enhances system reliability, simplifies maintenance, and facilitates future upgrades.

7.1 Microcontroller Unit

The system is built around the Arduino Uno, which is based on the ATmega328P microcontroller. Acting as the central processing unit of the system, it receives data from the sensors and user interface, processes the information according to the control algorithm, and generates the required control signals for the actuator and output devices.
The main specifications of the microcontroller include:

1. 8-bit AVR RISC architecture.
2. 16 MHz operating clock.
3. 32 KB Flash memory.
4. 2 KB SRAM.
5. 1 KB EEPROM for storing user settings when required.
6. 14 digital I/O pins and 6 analog input pins, providing sufficient interfaces for all hardware components.

7.2 Timing and Control Module

To ensure accurate execution of scheduled feeding operations, the system incorporates a DS1307 Real-Time Clock (RTC) module. The RTC maintains the correct date and time even during power interruptions using its built-in backup battery.
The module communicates with the microcontroller through the I²C protocol using pins A4 (SDA) and A5 (SCL). These communication lines are shared with the LCD display, reducing the total number of required I/O pins while maintaining reliable communication between devices.

7.3 Actuation Unit

Food dispensing is performed using an SG90 Servo Motor, which controls the opening and closing of the dispensing gate. The servo receives PWM control signals from the Arduino through digital pin D9, allowing accurate positioning for precise food portion delivery during both scheduled and manual feeding operations.

7.4 Sensing Unit

The system employs an HC-SR04 Ultrasonic Sensor to monitor food availability inside the storage container. The sensor measures the distance between itself and the food surface to estimate the remaining food level in real time. If the measured distance exceeds the predefined threshold of 20 cm, the system interprets the container as having a low food level and activates the warning indicators.

7.5 User Interface

The user interface enables convenient interaction with the system through both input and output devices, including:

1. A 4×4 Matrix Keypad for initiating manual feeding and user commands.
2. A 16×2 LCD display with an I²C (PCF8574) interface for displaying the current time, system status, feeding messages, and low-food warnings while minimizing microcontroller pin usage.
3. An Active Piezo Buzzer that generates audible alerts during feeding cycles and low-food conditions.
4. LED indicators, consisting of a Red LED for low-food warnings and a Green LED permanently connected to the regulated 5 V supply as a power indicator.

7.6 Power Supply Unit

The entire system operates from a regulated 5 V DC power supply that provides sufficient current for the microcontroller, sensors, display, and actuator. A common ground is shared among all hardware modules to ensure signal integrity, reduce electrical noise, and improve overall system stability during operation. A stable 5 V supply is particularly important to ensure reliable operation of the servo motor during food dispensing.

8. Electronic schematic diagram

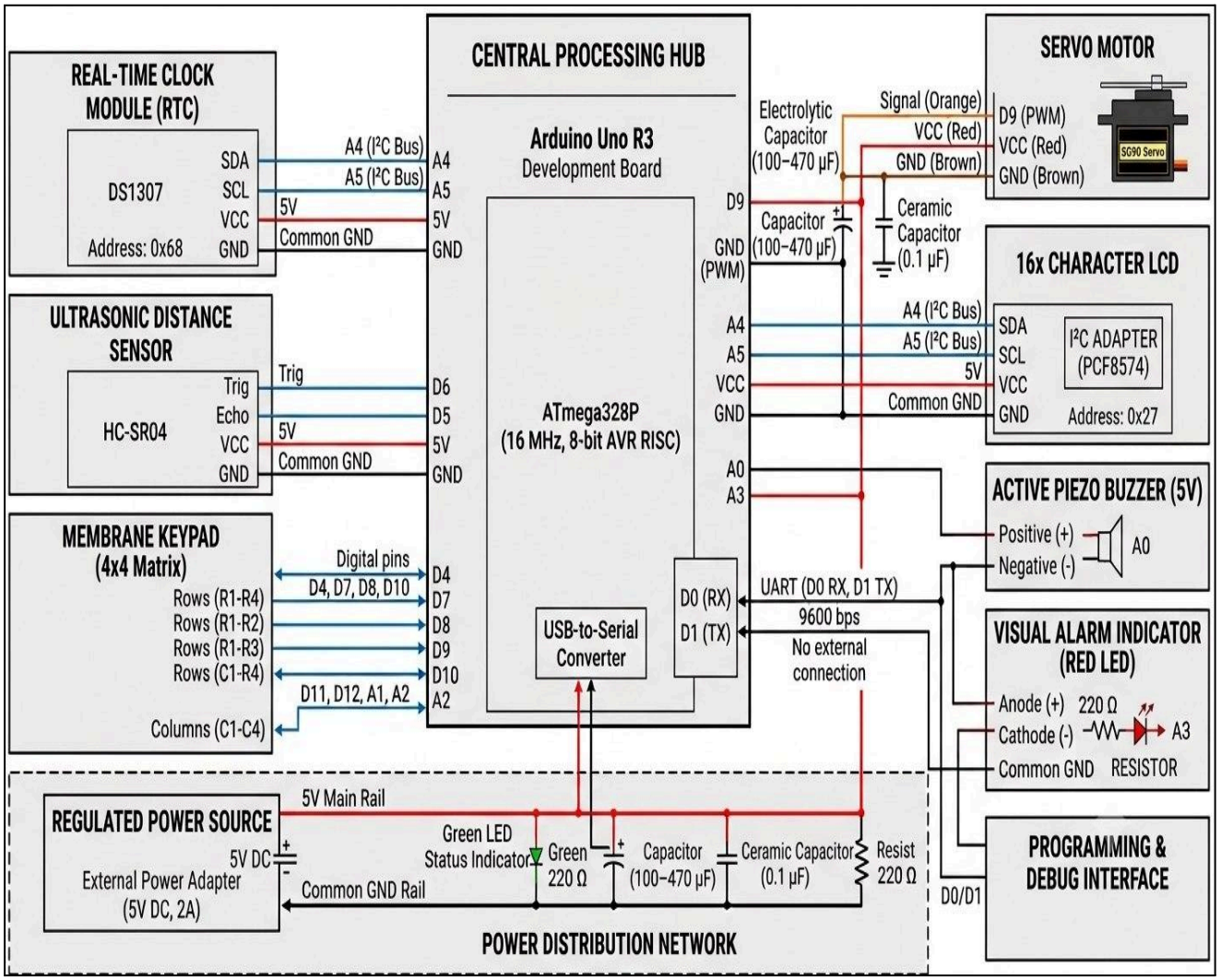

Figure 8.1 : Electronic Schematic Diagram of the Automatic Pet Feeding System

9. Components Selection

This section presents the engineering justification for selecting the hardware components used in the Automatic Pet Feeding System. The selected components provide an optimal balance between functionality, reliability, simplicity, cost-effectiveness, and availability while satisfying all project requirements.

9.1 Microcontroller: Arduino Uno (ATmega328P)

The Arduino Uno was selected as the main controller because it provides sufficient processing capability and enough digital and analog I/O pins to interface with all peripherals used in the system.
Compared with alternatives such as the Raspberry Pi or Arduino Mega, the Arduino Uno offers lower cost, lower power consumption, easier programming, and excellent community support, making it highly suitable for embedded automation applications.

9.2 Timekeeping Module: Real-Time Clock (RTC DS1307)

The DS1307 Real-Time Clock (RTC) module was selected to provide accurate and continuous timekeeping for scheduled feeding operations. Unlike software timers, the RTC maintains precise time even during power interruptions using its backup battery. It communicates with the Arduino through the I²C interface, requiring only two communication lines (SDA and SCL), thereby minimizing GPIO usage.

9.3 Actuator: SG90 Servo Motor

The SG90 micro servo motor was selected to operate the food dispensing gate because it provides precise angular positioning with simple PWM control. Unlike conventional DC motors, the servo can accurately rotate to predetermined angles (typically from 0° to 90° in this application), ensuring reliable opening and closing of the dispensing mechanism without requiring complex motor driver circuitry.

9.4 Food-Level Sensor: HC-SR04 Ultrasonic Sensor

The HC-SR04 ultrasonic sensor was selected to monitor the remaining food level inside the storage container. It performs non-contact distance measurements using ultrasonic waves, ensuring hygienic operation without touching the food. Compared with infrared sensors, ultrasonic sensing is less affected by food color, dust, or ambient lighting conditions, resulting in more stable and reliable measurements.

9.5 User Display: LCD 16×2 with I²C Module

A 16×2 LCD equipped with an I²C interface was selected to display important system information such as the current time, feeding status, and low-food warnings. The I²C interface significantly reduces wiring complexity by requiring only two communication pins while allowing the LCD to share the communication bus with the RTC module.

9.6 Input Interface: 4×4 Matrix Keypad

A 4×4 matrix keypad was selected to provide a convenient user interface for initiating manual feeding. The keypad enables direct user interaction, making system operation intuitive and responsive while requiring only eight microcontroller pins.

9.7 Notification System: Active Buzzer and LEDs

An active piezo buzzer, together with a red LED and a green LED, was selected to provide audible and visual feedback. The buzzer and red LED notify the user of feeding operations and low-food conditions, while the green LED serves as a continuous power indicator. These indicators require minimal hardware resources while improving system usability.

9.8 Bill of Materials (BOM)

Component Model Quantity Primary Function

Component	Model	Quantity	Primary Function
Main Controller	Arduino Uno R3 (ATmega328P)	1	System control and processing
Real-Time Clock	DS1307 RTC Module	1	Accurate timekeeping
Display	LCD 16×2 with I²C Module	1	User interface
Actuator	SG90 Micro Servo Motor	1	Food dispensing mechanism
Food-Level Sensor	HC-SR04 Ultrasonic Sensor	1	Food level monitoring
User Input	4×4 Matrix Keypad	1	Manual feeding input
Audible Indicator	Active Piezo Buzzer	1	Audio notifications
Visual Indicators	Red LED, Green LED	2	Status indication
Current-Limiting Resistors	220 Ω	2	LED protection
Power Supply	5 V / 2 A DC Adapter	1	System power supply

10. Circuit Description

This section describes the electrical connections and signal routing between the Arduino Uno and all peripheral modules used in the Automatic Pet Feeding System. The circuit has been designed to ensure reliable communication, proper power distribution, stable operation, and ease of maintenance.

10.1 Central Controller and Power Distribution

The Arduino Uno (ATmega328P) serves as the central controller of the system, coordinating communication between all sensors, input devices, indicators, and the actuator. The entire circuit operates at a regulated 5 V logic level.
A regulated 5 V / 2 A DC external power supply provides stable power to the system. Since the SG90 servo motor may draw peak current during movement, maintaining a stable 5 V supply ensures reliable operation. The Arduino and all peripheral modules share a common GND reference to ensure signal integrity and stable system performance.

10.2 I²C Communication Bus

To reduce wiring complexity and minimize the number of occupied I/O pins, both the DS1307 Real-Time Clock (RTC) module and the 16×2 LCD display communicate through the I²C bus.
SDA (Serial Data) → Arduino A4
SCL (Serial Clock) → Arduino A5
Both devices receive 5 V and GND while operating independently through their unique I²C addresses (0x68 for the RTC and 0x27 for the LCD).

10.3 Servo Motor Control Circuit

The SG90 servo motor is responsible for opening and closing the food dispensing gate. The motor requires three electrical connections:

1. 5 V power supply
2. Common Ground (GND)
3. PWM control signal connected to Arduino digital pin D9

Digital pin D9 was selected because it supports hardware PWM, enabling accurate angular positioning during feeding operations.

10.4 Ultrasonic Sensor Circuit (HC-SR04)

The HC-SR04 ultrasonic sensor continuously measures the distance to estimate the remaining food level inside the storage container. Its electrical connections are:
VCC → 5 V
GND → GND
TRIG → Arduino D6
ECHO → Arduino D5
The measured distance is processed by the Arduino to determine whether the food level is above or below the predefined threshold.

10.5 User Input Interface (4×4 Matrix Keypad)

The 4×4 matrix keypad provides manual user input. The pin allocation matches the firmware configuration implemented in the Arduino program:
Rows (R1–R4): D4, D7, D8, D10
Columns (C1–C4): D11, D12, A1, A2
This configuration allows the Arduino to scan the keypad efficiently while avoiding pin conflicts with other peripherals.

10.6 Notification and Status Indicators

The notification system provides both audible and visual feedback.
Active Piezo Buzzer: Positive terminal connected to A0, negative terminal connected to GND.
Red LED (Low-Food Warning): Anode connected to A3 through a 220 Ω current-limiting resistor, cathode connected to GND.
Green LED (Power Indicator): Permanently connected to the regulated 5 V supply through a 220 Ω current-limiting resistor to indicate system power.

10.7 Pin Assignment Summary

Arduino Pin Connected Device Function

Arduino pin	Connected devices	Function
D4	Keypad	Row 1 Control
D5	HC-SR04 Echo	Distance Echo Signal
D6	HC-SR04 Trigger	Distance Trigger Signal
D7	Keypad R2	Row 2 Control
D8	Keypad R3	Row 3 Control
D9	SG90 Servo	PWM Gate Control
D10	Keypad R4	Row 4 Control
D11	Keypad C1	Column 1 Control
D12	Keypad C2	Column 2 Control
A0	Active Piezo Buzzer	Audio Warning

A1	Keypad C3	Column 3 Control
A2	Keypad C4	Column 4 Control
A3	Red LED	Low-Food Warning
A4	LCD + RTC	I²C SDA
A5	LCD + RTC	I²C SCL

10.8 Circuit Reliability and Design Considerations

Arduino pins D0 (RX) and D1 (TX) are intentionally left unused to preserve serial programming and debugging capability.
Each Arduino pin is assigned to a single hardware function, eliminating pin conflicts.
All modules operate at compatible 5 V logic levels and share a common GND reference.
The pin allocation was verified using Wokwi simulation to ensure conflict-free operation before software deployment.
The shared I²C bus minimizes wiring complexity while allowing reliable communication between the RTC and LCD modules.

11. Power Supply Design

A reliable power supply is fundamental to the stable operation of the proposed Automatic Pet Feeding System. Stable voltage regulation ensures dependable operation of the microcontroller, sensors, actuator, and peripheral modules while preventing unexpected resets and communication errors. This section describes the power architecture, estimated power consumption, safety margin, and electrical stabilization techniques adopted in the system.

11.1 Power Supply Architecture

The entire system operates from a regulated 5 V DC external power supply rated at 2 A. The positive output of the power supply is connected to the main 5 V power rail, while all hardware modules share a common Ground (GND) reference to ensure proper signal integrity and reliable communication.
The SG90 servo motor is supplied from the regulated 5 V power rail. In a practical hardware implementation, a dedicated regulated 5 V supply is recommended for the servo motor to prevent voltage drops caused by its startup current and to improve overall system stability during dispensing operations.

11.2 Power Consumption Analysis

An estimated power budget was prepared to verify that the selected power supply is capable of operating all hardware modules safely under both normal and peak operating conditions.

Component	**Operating Voltage**	**Typical Current**	**Peak Currunt**
Arduino Uno	5V	50 mA	80 mA
DS1307 RTC	5V	1 mA	2 mA
LCD 16×2 with I²C	5V	20 mA	30 mA
HC-SR04 Ultrasonic Sensor	5V	15 mA	20 mA
4×4 Matrix Keypad	5V	<1 mA	1 mA
Active Piezo Buzzer	5V	30 mA	40 mA
Red & Green LEDs	5V	20 mA	20 mA
SG90 Servo Motor	5V	100 mA	650 mA
Total	**5V**	**≈237 mA**	**≈843 mA**

11.3 Safety Margin and Brownout Prevention

The maximum estimated current consumption of approximately 843 mA represents less than half of the rated output current of the selected 5 V / 2 A power supply. This substantial safety margin helps prevent excessive voltage drops during transient conditions, particularly when the servo motor starts moving.

Maintaining a stable 5 V supply minimizes the possibility of brownout conditions, prevents unexpected Arduino resets, and ensures uninterrupted execution of scheduled and manual feeding operations.

11.4 Noise Reduction and Power Stabilization

To further improve electrical stability, a bulk electrolytic capacitor in the range of 100–470 µF is recommended across the servo motor power supply to compensate for transient current demands during startup.

Additionally, 0.1 µF ceramic decoupling capacitors should be placed close to the Arduino Uno and other sensitive electronic modules to suppress high-frequency electrical noise, improve signal integrity, and enhance the overall reliability of the system.

Note: The current values presented above are typical engineering estimates and may vary slightly depending on operating conditions, component tolerances, and servo motor loading.

12. Sensor and Actuator Interfaces

This section describes the hardware and software interfaces between the Arduino Uno (ATmega328P) microcontroller and the sensors and actuators used in the Automatic Pet Feeding System. It explains how measurement data are acquired, how control signals are generated, and how communication between different hardware modules is achieved to ensure reliable and accurate system operation.

12.1 Ultrasonic Sensor Interface

The remaining food level inside the storage container is monitored using an HC-SR04 ultrasonic sensor.
The Arduino periodically transmits a trigger pulse to the sensor and measures the duration of the returning echo signal to calculate the distance between the sensor and the food surface. The measured distance is then compared with a predefined threshold (20 cm) to determine whether the food level is sufficient.
The sensor connections are:
TRIG → D6
ECHO → D5
Whenever the measured distance exceeds the predefined threshold, the system identifies a low-food condition and activates the warning indicators.

12.2 I²C Communication Interface

The DS1307 Real-Time Clock (RTC) module and the 16×2 LCD display communicate with the Arduino Uno through the I²C communication bus.
The communication lines are assigned as follows:
SDA → A4
SCL → A5
Since each device has a unique I²C address, both modules communicate simultaneously over the same two communication lines without interference. This approach reduces wiring complexity and preserves additional input/output pins for other peripherals.

12.3 Matrix Keypad Interface

A 4×4 matrix keypad provides the primary user input interface for the system.
The keypad connections are:
Rows: D4, D7, D8, D10
Columns: D11, D12, A1, A2
The Arduino continuously scans the keypad matrix using the Keypad library to detect user inputs. In the implemented firmware, pressing the A key initiates a manual feeding cycle.

12.4 Servo Motor Interface

The food dispensing mechanism is driven by an SG90 micro servo motor connected to digital pin D9.
The Arduino controls the servo position using PWM signals, allowing accurate opening and closing of the dispensing gate during both scheduled and manual feeding operations.
This method provides smooth, repeatable motion while maintaining simple hardware implementation and reliable operation.

12.5 Buzzer and LED Interfaces

An active piezo buzzer connected to A0 generates audible notifications during feeding operations and whenever the food level becomes critically low.
The system also incorporates two LED indicators to provide visual feedback.
The Red LED, connected to A3, indicates low-food conditions and warning states.
The Green LED is connected to the regulated 5 V power supply through a current-limiting resistor and remains illuminated whenever the system is powered, serving as a continuous power indicator.
Together, these visual and audible indicators improve user awareness and simplify system monitoring during normal operation.

13. Communication Interfaces and Data Transfer Protocols

This section presents the communication interfaces and data transfer protocols employed in the Automatic Pet Feeding System. The selected interfaces ensure reliable communication between the Arduino Uno (ATmega328P) and the peripheral modules while minimizing wiring complexity, avoiding pin conflicts, and maintaining efficient data exchange.

13.1 I²C Communication Bus

The I²C bus serves as the primary communication interface between the Arduino Uno (Master), the DS1307 Real-Time Clock (RTC) module, and the 16×2 LCD display equipped with a PCF8574 I²C interface, both operating as slave devices.
Physical Connections
SDA (Serial Data) → Arduino A4
SCL (Serial Clock) → Arduino A5
Both communication lines share the same I²C bus. The required pull-up resistors are typically integrated into the DS1307 RTC and LCD interface modules, allowing reliable communication without additional external components.
Bus Characteristics
Communication mode: Synchronous serial communication
Standard operating speed: 100 kHz
Addressing scheme: 7-bit addressing
Typical device addresses:
DS1307 RTC → 0x68
PCF8574 LCD Interface → 0x27
The unique address assigned to each device enables multiple peripherals to communicate simultaneously over the same two communication lines without interference.

13.2 GPIO Timing Interface (HC-SR04 Ultrasonic Sensor)

The HC-SR04 ultrasonic sensor communicates with the Arduino using two dedicated digital GPIO pins.
TRIG → D6
ECHO → D5
The Arduino initiates a measurement by generating a 10 μs trigger pulse. The sensor returns an Echo pulse whose duration is proportional to the round-trip travel time of the ultrasonic wave. The measured pulse width is processed to calculate the distance between the sensor and the food surface.

13.3 PWM Control Interface (SG90 Servo Motor)

The SG90 servo motor is controlled using the Arduino hardware PWM output.
PWM Output → D9
The Arduino generates periodic PWM control pulses at approximately 50 Hz, where the pulse width determines the angular position of the servo shaft. This method enables accurate and repeatable control of the food dispensing mechanism during both scheduled and manual feeding operations.

13.4 UART Serial Communication

The ATmega328P microcontroller includes a built-in hardware UART interface.
RX → D0
TX → D1
These pins are intentionally reserved for program uploading and serial debugging through the USB interface. No external hardware is connected to D0 or D1, preventing communication conflicts.
During development and simulation, the UART interface operates at 9600 bps and is used to monitor system status, display sensor readings, and assist in software debugging through the Serial Monitor.

13.5 Summary of Communication Interfaces

Communication Interface	Protocol	Arduino Pins	Connected Device	Data Direction	Operating Speed
I²C Bus	Synchronous Serial	A4 (SDA), A5 (SCL)	DS1307 RTC, LCD (PCF8574)	Bidirectional	100 kHz
Ultrasonic Interface	GPIO Pulse Timing	D6, D5	HC-SR04	Bidirectional	10 µs Trigger Pulse
Servo Control	PWM	D9	SG90 Servo Motor	Arduino → Servo	50 Hz
UART Serial	Asynchronous Serial	D0 (RX), D1 (TX)	PC / Serial Monitor	Bidirectional	9600 bps

14. Software Flowchart

The software architecture of the Automatic Pet Feeding System is organized as a continuous control loop executed by the Arduino Uno. The program initializes all hardware modules, continuously monitors the system status, updates the user interface, checks scheduled feeding times, and responds to manual feeding requests. Figure 14.1 illustrates the overall software flow of the system.

14.1 Software Operation

The software executes the following sequence:
Start the program.
Initialize the Arduino Uno and configure all input/output pins.
Initialize the LCD display and the DS1307 Real-Time Clock (RTC) module.
Initialize the ultrasonic sensor, servo motor, buzzer, LED indicator, and keypad.
Enter the continuous main loop.
Read the current time from the RTC module.
Measure the food level using the HC-SR04 ultrasonic sensor.
Display the current time and food status on the LCD.
Compare the measured distance with the predefined threshold (20 cm).
If the food level is low, activate the red LED and buzzer; otherwise, deactivate the warning indicators.
Check whether the current time matches one of the scheduled feeding times (08:00 or 18:00).
If a scheduled feeding time is reached and feeding has not already occurred during the current minute, execute the food dispensing routine and update the feeding status.
Monitor the keypad for user input.
If the A key is pressed, execute the manual food dispensing routine.
Return to the beginning of the main loop and repeat continuously.

14.2 Flowchart Description

The software flowchart consists of three major functional stages:
System Initialization: Initializes all hardware modules, including the LCD, RTC, ultrasonic sensor, servo motor, keypad, buzzer, and LED indicators.
Continuous Monitoring: Continuously reads the current time, measures the food level, updates the LCD display, and activates warning indicators whenever the food level falls below the predefined threshold.
Feeding Control: Executes automatic feeding according to the predefined schedule or manual feeding when the user presses the A key on the keypad. The program uses an internal software flag to prevent repeated feeding during the same scheduled minute.

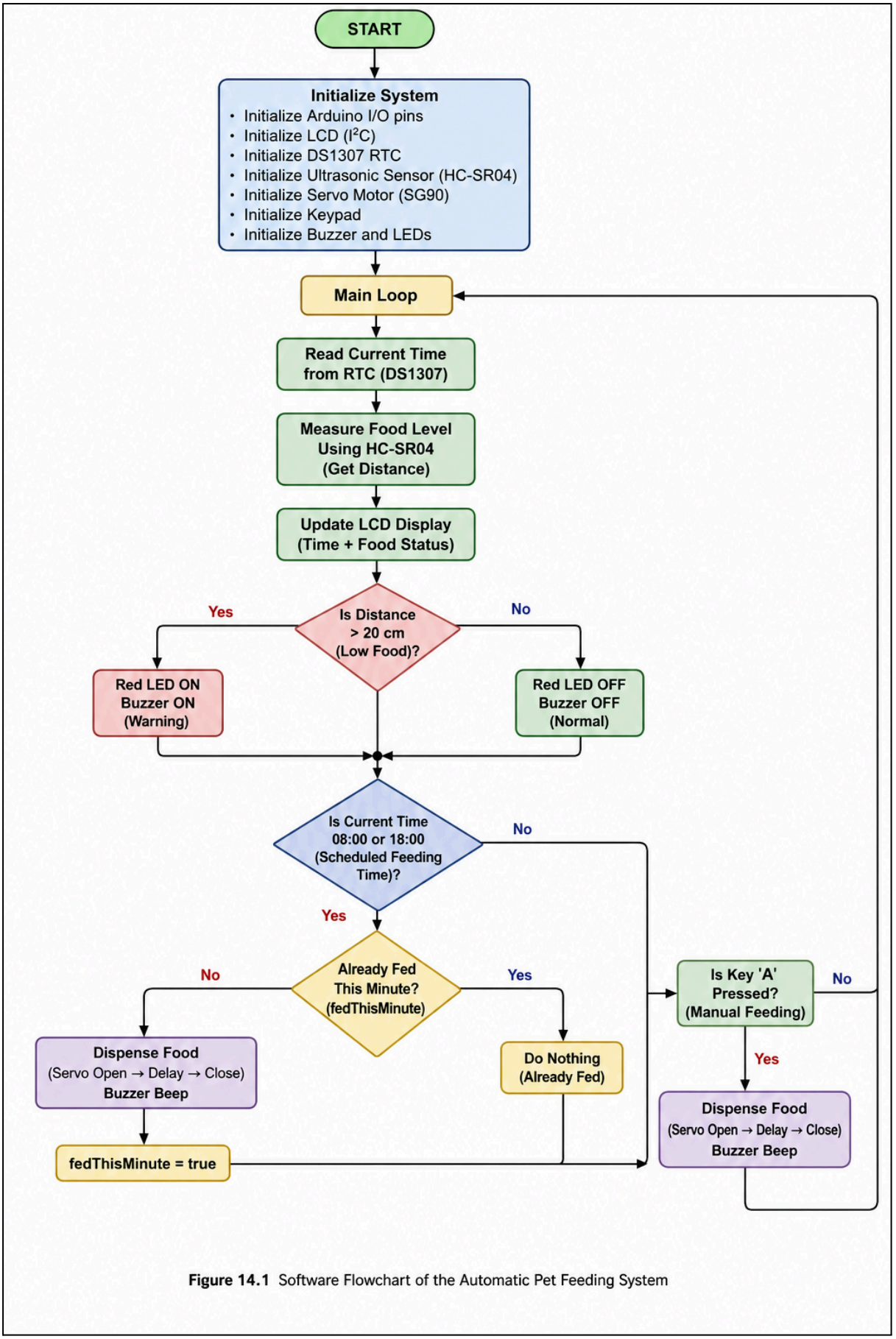

Figure 14.1 Software Flowchart of the Automatic Pet Feeding System

15. Software Architecture

The software architecture of the Automatic Pet Feeding System is designed using a modular and sequential programming approach implemented on the Arduino Uno (ATmega328P) platform. The software integrates all hardware modules into a single control program that continuously monitors the system status, performs scheduled feeding operations, responds to manual user commands, and updates the user interface.
The program follows a continuous execution model based on the Arduino setup() and loop() functions. During system startup, all hardware peripherals are initialized. Once initialization is complete, the software enters an infinite execution loop that repeatedly performs sensing, decision-making, user interaction, and actuator control.

15.1 System Initialization Module

The initialization module is implemented in the setup() function. It prepares all hardware resources before normal operation begins.
The initialization process includes:

- Initializing serial communication for debugging.
- Initializing the 16×2 I²C LCD display.
- Initializing the DS1307 Real-Time Clock (RTC) module.
- Configuring the ultrasonic sensor input and output pins.
- Configuring the buzzer and LED output pins.
- Initializing the SG90 servo motor and setting the dispensing gate to the closed position.
- Displaying the system startup message.

After successful initialization, the program enters the continuous execution loop.

15.2 Monitoring and Decision-Making Module

The monitoring module executes continuously inside the loop() function.
Its primary responsibilities include:

- Reading the current time from the DS1307 RTC.
- Measuring the food level using the HC-SR04 ultrasonic sensor.
- Updating the LCD with the current time and food status.
- Comparing the measured distance with the predefined threshold (20 cm).
- Activating or deactivating the warning indicators according to the measured food level.

This module continuously supervises the operating condition of the system and provides real-time feedback to the user.

15.3 Automatic Feeding Module

The automatic feeding module performs scheduled feeding based on the current time obtained from the RTC.
The implemented feeding schedule consists of:
08:00 – First feeding cycle.
18:00 – Second feeding cycle.
To prevent repeated food dispensing within the same minute, the software uses an internal Boolean flag (fedThisMinute). Once feeding is completed, the flag is set until the scheduled minute has elapsed.

15.4 Manual Feeding Module

The manual feeding module continuously monitors the 4×4 matrix keypad using the Keypad library. When the user presses the A key, the software immediately initiates a manual feeding cycle by calling the food dispensing function. This allows feeding to be performed independently of the predefined schedule.

15.5 Food Dispensing Module

Food dispensing is implemented through the dedicated dispenseFood() function.
This function performs the following sequence:
1. Displays a feeding message on the LCD.
2. Generates a short buzzer tone.
3. Rotates the SG90 servo motor to 90° to open the dispensing gate.
4. Waits for approximately 2 seconds to allow food to be released.
5. Returns the servo motor to 0° to close the gate.
6. Clears the display and returns control to the main program.
Using a dedicated function eliminates code duplication because the same routine is called for both scheduled and manual feeding operations.

15.6 Software Libraries

The software relies on several Arduino libraries to simplify hardware interfacing and improve program reliability.

Library	Purpose
Wire	communication between the Arduino, RTC, and LCD
LiquidCrystal_I2C	Control of the 16×2 LCD display
RTClib	Communication with the DS1307 Real-Time Clock
Servo	PWM control of the SG90 servo motor
Keypad	Scanning and reading the 4×4 matrix keypad

15.7 Software Architecture Summary

The software architecture follows a modular design in which each functional module performs a specific task while cooperating within a single continuous execution loop. This approach simplifies software maintenance, improves readability, minimizes code duplication through reusable functions, and provides reliable integration of sensing, monitoring, user interaction, and automatic control, making it well suited for embedded systems based on the Arduino Uno.

16. Program Functions

This section describes the main software functions implemented in the Automatic Pet Feeding System. The program is developed using the Arduino programming environment and follows the standard Arduino execution model consisting of the setup() and loop() functions, together with a dedicated function for food dispensing.

16.1 setup() Function

The setup() function is executed once when the Arduino is powered on or reset. Its purpose is to initialize all hardware modules and prepare the system for normal operation.

The initialization process includes:

- Starting serial communication at 9600 bps.
- Initializing the 16×2 I²C LCD display.
- Initializing the DS1307 Real-Time Clock (RTC) module.
- Configuring the ultrasonic sensor pins.
- Configuring the buzzer and LED output pins.
- Attaching the SG90 servo motor to digital pin D9.
- Setting the servo to its initial closed position.
- Displaying the startup message on the LCD.

Once initialization is complete, control is transferred to the continuous execution loop.

16.2 loop() Function

The loop() function executes continuously throughout system operation.
During each iteration, the program performs the following tasks:
1. Reads the current time from the DS1307 RTC module.
2. Measures the food level using the HC-SR04 ultrasonic sensor.
3. Updates the LCD with the current time and food status.
4. Compares the measured distance with the predefined threshold (20 cm).
5. Activates or deactivates the buzzer and the red LED according to the food level.
6. Checks whether the current time matches one of the scheduled feeding times (08:00 or 18:00).
7. Prevents repeated feeding during the same minute using the fedThisMinute flag.
8. Monitors the keypad for manual feeding requests.
9. Executes a manual feeding cycle when the A key is pressed.
This continuous loop enables real-time monitoring and automatic system control.

16.3 dispenseFood() Function

The dispenseFood() function performs the complete food dispensing sequence.
The function executes the following operations:
1. Displays the "Feeding Pet..." message on the LCD.
2. Generates a short audible notification using the buzzer.
3. Rotates the SG90 servo motor to 90° to open the dispensing gate.
4. Waits for approximately 2 seconds to allow food to be dispensed.

5. Rotates the servo back to 0° to close the dispensing gate
6. Clears the LCD display before returning control to the main program.
The same function is called by both the scheduled feeding routine and the manual feeding routine, ensuring consistent system behavior while avoiding code duplication.

16.4 Program Variables

Several global variables are used to control system operation.

Variable	Purpose
feedHour1	First scheduled feeding hour (08:00)
feedMinute1	Minute of the first feeding schedule
feedHour2	Second scheduled feeding hour (18:00)
feedMinute2	Minute of the second feeding schedule
emptyDistanc	Distance threshold (20 cm) used to detect a low-food condition
fedThisMinut	Prevents repeated feeding within the same scheduled minute
distance	Stores the measured distance from the ultrasonic sensor
duration	Stores the echo pulse duration measured by the ultrasonic sensor
key	Stores the key pressed on the keypad

16.5 Overall Program Operation

The software begins by initializing all hardware modules in the setup() function. It then enters the loop() function, where it continuously monitors the current time, measures the food level, updates the LCD display, checks scheduled feeding times, and monitors the keypad for manual feeding requests.
Whenever a scheduled feeding time is reached or the user presses the A key, the dispenseFood() function is executed to operate the servo motor, dispense food, generate an audible notification, and update the display.
This modular organization improves program readability, simplifies maintenance, and ensures reliable operation of the Automatic Pet Feeding System while remaining fully consistent with the implemented Arduino code.

17. Complete Source Code

This chapter presents the complete Arduino source code developed for the Automatic Pet Feeding System. The software integrates all hardware modules into a single application that performs system initialization, real-time monitoring, scheduled feeding, manual feeding, user notification, and actuator control.

The program was developed using the Arduino IDE and tested through Wokwi simulation to verify the correctness of the hardware connections and software operation before deployment.

The source code incorporates the following software libraries:

- Wire – I²C communication.
- LiquidCrystal_I2C – LCD display control.
- RTClib – DS1307 Real-Time Clock communication.
- Servo – SG90 servo motor control.
- Keypad – 4×4 matrix keypad interface.

The implemented software performs the following functions:

- Initializes all hardware peripherals during system startup.
- Reads the current time from the DS1307 RTC module.
- Monitors the food level using the HC-SR04 ultrasonic sensor.
- Displays the current time and system status on the 16×2 I²C LCD.
- Activates the red LED and buzzer whenever a low-food condition is detected.
- Executes automatic feeding at the predefined schedule (08:00 and 18:00).
- Supports manual feeding through the A key on the 4×4 matrix keypad.
- Controls the SG90 servo motor to open and close the dispensing gate during feeding operations.
- Prevents repeated automatic feeding within the same minute using the fedThisMinute software flag.

The complete Arduino source code is provided in the following pages.

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <RTClib.h>
#include <Servo.h>
#include <Keypad.h>
#define TRIG_PIN 6
#define ECHO_PIN 5
#define SERVO_PIN 9
#define BUZZER_PIN A0
#define RED_LED_PIN A3
LiquidCrystal_I2C lcd(0x27,16,2);
RTC_DS1307 rtc;
Servo feederServo;
const byte ROWS=4;
const byte COLS=4;
char keys[ROWS][COLS]={
{'1','2','3','A'},
{'4','5','6','B'},
{'7','8','9','C'},
{'*','0','#','D'}
};
byte rowPins[ROWS]={4,7,8,10};
byte colPins[COLS]={11,12,A1,A2};
Keypad keypad=Keypad(makeKeymap(keys),rowPins,colPins,ROWS,COLS);
const byte feedHour1=8;
const byte feedMinute1=0;
```

```
const byte feedHour2=18;
const byte feedMinute2=0;
const float emptyDistance=20.0;
long duration;
float distance;
bool fedThisMinute=false;
void measureFoodLevel();
void updateDisplay(DateTime now);
void checkFoodLevel();
void checkAutomaticFeeding(DateTime now);
void checkManualFeeding();
void dispenseFood();
void setup()
{
Serial.begin(9600);
lcd.init();
lcd.backlight();
Wire.begin();
rtc.begin();
if(!rtc.isrunning())
rtc.adjust(DateTime(F(DATE),F(TIME)));
pinMode(TRIG_PIN,OUTPUT);
pinMode(ECHO_PIN,INPUT);
pinMode(BUZZER_PIN,OUTPUT);
pinMode(RED_LED_PIN,OUTPUT);
digitalWrite(RED_LED_PIN,LOW);
feederServo.attach(SERVO_PIN);
feederServo.write(0);
lcd.clear();
lcd.setCursor(0,0);
lcd.print("Automatic Pet");
lcd.setCursor(0,1);
lcd.print("Feeder System");
delay(2000);
lcd.clear();
}
void loop()
{
DateTime now = rtc.now();
measureFoodLevel();
updateDisplay(now);
checkFoodLevel();
checkAutomaticFeeding(now);
checkManualFeeding();
delay(200);
}
void measureFoodLevel()
{
digitalWrite(TRIG_PIN, LOW);
delayMicroseconds(2);
digitalWrite(TRIG_PIN, HIGH);
delayMicroseconds(10);
digitalWrite(TRIG_PIN, LOW);
duration = pulseIn(ECHO_PIN, HIGH);
distance = duration * 0.0343 / 2.0;
}
void updateDisplay(DateTime now)
{
lcd.setCursor(0,0);
```

```
char timeBuffer[17];
sprintf(timeBuffer,"%02d:%02d:%02d",
now.hour(),
now.minute(),
now.second());
lcd.print("Time:");
lcd.print(timeBuffer);
lcd.print(" ");
lcd.setCursor(0,1);
if(distance > emptyDistance)
{
lcd.print("Food: LOW ");
}
else
{
lcd.print("Food: OK ");
}
}
void checkFoodLevel()
{
if (distance > emptyDistance)
{
digitalWrite(RED_LED_PIN, HIGH);
tone(BUZZER_PIN, 1000);
}
else
{
digitalWrite(RED_LED_PIN, LOW);
noTone(BUZZER_PIN);
}
}
void checkAutomaticFeeding(DateTime now)
{
bool feedingTime =
(now.hour() == feedHour1 && now.minute() == feedMinute1) ||
(now.hour() == feedHour2 && now.minute() == feedMinute2);
if (feedingTime)
{
if (!fedThisMinute)
{
dispenseFood();
fedThisMinute = true;
}
}
else
{
fedThisMinute = false;
}
}
void checkManualFeeding()
{
char key = keypad.getKey();
if (key == 'A')
{
dispenseFood();
}
}
void dispenseFood()
{
```

```
noTone(BUZZER_PIN);
lcd.clear();
lcd.setCursor(0,0);
lcd.print("Feeding Pet...");
lcd.setCursor(0,1);
lcd.print("Please Wait");
tone(BUZZER_PIN,2000,300);
feederServo.write(90);
delay(2000);
feederServo.write(0);
delay(500);
lcd.clear();
}
```

18. Simulation: Testing and Validation

This chapter presents the testing and validation procedures performed to verify the correct functionality, reliability, and performance of the **Automatic Pet Feeding System**. Since physical prototyping can introduce hardware variations and debugging complexities, the entire system was comprehensively simulated and tested using the **Wokwi Online Simulator**. The verification process focused on validating hardware connections, software logic, sensor operation, actuator response, and user interface interactions under various operating scenarios.

18.1 Testing Objectives

The primary objectives of the simulation testing were to:
Verify that all peripheral modules (**DS1307 RTC**, **HC-SR04 ultrasonic sensor**, **SG90 servo motor**, **16×2 I²C LCD**, **4×4 matrix keypad**, **buzzer**, and **LED indicators**) operate correctly without communication conflicts or pin allocation errors.
Validate the real-time clock synchronization and ensure scheduled feeding operations are executed precisely at the predefined times (**08:00** and **18:00**).
Test the ultrasonic distance measurement and verify that the low-food warning (**Red LED** and **Buzzer**) is activated whenever the measured distance exceeds the predefined **20 cm** threshold, indicating a low-food condition.
Confirm the responsiveness of manual feeding initiated by pressing the **A** key on the **4×4 matrix keypad**.
Verify the correct operation of the software protection mechanism (**fedThisMinute**) to prevent repeated food dispensing during the same scheduled minute.

18.2 Functional Testing Scenarios and Results

To ensure comprehensive validation, four primary testing scenarios were performed during the simulation.

1. Scheduled Feeding Test

Procedure:
The **DS1307 RTC** was adjusted so that the system time approached one of the scheduled feeding times (**08:00** or **18:00**).
Expected Result:
The system should automatically execute the food dispensing routine when the scheduled time is reached.
Observed Result:
At the scheduled time, the LCD displayed **"Feeding Pet..."**, the buzzer generated a short audible tone, the **SG90 servo motor** rotated smoothly to **90°** for approximately **2 seconds** to open the dispensing gate, and then returned to **0°** to close it. The **fedThisMinute** flag successfully prevented repeated feeding during the same minute.

2. Food-Level Monitoring and Alert Test

Procedure:
The simulated distance measured by the **HC-SR04 ultrasonic sensor** in Wokwi was adjusted to values above and below the predefined **20 cm** threshold.
Expected Result:
When the measured distance exceeds **20 cm**, the system should indicate a low-food condition.
When the measured distance is **20 cm or less**, the system should indicate normal operation.
Observed Result:
When the measured distance exceeded **20 cm**, the LCD immediately displayed **"Food: LOW!"**, the **Red LED** turned ON, and the **buzzer** generated a continuous warning tone. When the measured

distance returned to **20 cm or below**, the warning indicators were automatically deactivated, and the LCD displayed **"Food: OK"**.

3. Manual Feeding Test

Procedure:
During normal system operation, the user pressed the **A** key on the **4×4 matrix keypad**.
Expected Result:
The system should immediately execute the **dispenseFood()** routine while continuing normal system operation.
Observed Result:
The system successfully detected the key press using the **Keypad** library, displayed the feeding message, activated the servo motor, and completed the feeding cycle without affecting RTC operation or overall system stability.

4. I²C Bus Communication Test

Procedure:
The shared **I²C** communication between the **DS1307 RTC (0x68)** and the **16×2 LCD with PCF8574 interface (0x27)** was monitored while both devices operated simultaneously on pins **A4 (SDA)** and **A5 (SCL)**.
Expected Result:
Both devices should communicate simultaneously without communication conflicts or data errors.
Observed Result:
The LCD continuously displayed the current time and system status while receiving real-time data from the RTC without communication delays or instability, confirming the reliability of the shared **I²C** bus architecture.

18.3 Interactive Wokwi Simulation Link

The fully functional, real-time interactive simulation of the Automatic Pet Feeding System is available online. The virtual circuit configuration, sensor responses, and code execution can be tested directly via the following link:
Live Wokwi Simulation: https://wokwi.com/projects/471244750730611713

18.4 Summary of Validation Outcomes

All simulation scenarios were completed successfully, demonstrating that the hardware configuration, electrical connections, and embedded software operated correctly as an integrated embedded system. The Wokwi simulation confirmed that the **Automatic Pet Feeding System** satisfies all functional and operational requirements, including accurate timekeeping, reliable food-level monitoring, scheduled and manual feeding, stable user interface operation, and dependable communication between all hardware modules. These results indicate that the proposed design is suitable for practical implementation following successful simulation validation.

19. Results and Discussion

This section presents a comprehensive evaluation of the Automatic Pet Feeding System based on the implementation and simulation results. The discussion focuses on system performance, sensing accuracy, communication reliability, user interaction, and engineering considerations identified during the validation process.

19.1 System Performance and Execution

The embedded control algorithm demonstrated stable and reliable performance throughout all simulated operating conditions.

Scheduled Dispensing Performance: The integration of the DS1307 Real-Time Clock (RTC) enabled accurate execution of the scheduled feeding operations at 08:00 and 18:00. The software successfully completed the programmed dispensing sequence, including opening the gate for approximately 2 seconds before returning it to the closed position.

Redundancy Protection: The internal software flag (fedThisMinute) successfully prevented repeated feeding operations during the same scheduled minute, eliminating unintended multiple dispensing cycles.

Actuation Reliability: The SG90 micro servo motor consistently performed smooth transitions between 0° (gate closed) and 90° (gate open). In practical hardware implementations, supplying the servo from a stable regulated 5 V power source is recommended to minimize voltage fluctuations during motor operation.

19.2 Sensor Accuracy and Low-Food Alert Verification

The HC-SR04 ultrasonic sensor provided reliable non-contact distance measurements for monitoring the food level inside the storage container.

Normal Operating Condition (Distance ≤ 20 cm): When the measured distance was 20 cm or less, the LCD displayed "Food: OK", the Red LED remained OFF, and the buzzer stayed inactive.

Low-Food Condition (Distance > 20 cm): When the measured distance exceeded 20 cm, the system immediately entered the warning state. The LCD displayed "Food: LOW!", the Red LED illuminated, and the active piezo buzzer generated a continuous audible alert.

The ultrasonic sensing method provided stable and repeatable distance measurements while remaining unaffected by ambient lighting conditions.

19.3 User Interface and Communication Reliability

I²C Communication Stability: Sharing the I²C communication bus (A4/SDA and A5/SCL) between the DS1307 RTC (0x68) and the PCF8574-based LCD (0x27) operated reliably without address conflicts or communication errors. The LCD continuously displayed the current time and system status while receiving real-time data from the RTC.

Manual Feeding Responsiveness: The 4×4 matrix keypad accurately detected user inputs through the Keypad library. Pressing the A key immediately initiated a manual feeding cycle without affecting the normal operation of the RTC or the continuous monitoring process.

19.4 Engineering Insights and Limitations

Design Strengths: The software is organized into clearly structured functional sections that simplify program readability, maintenance, and future expansion. The integration of sensing,

scheduling, user interaction, and actuator control provides reliable operation within a straightforward embedded software architecture.

System Limitations: The current design estimates the remaining food level using ultrasonic distance measurement rather than direct weight measurement. Although this approach effectively detects low-food conditions, it cannot identify mechanical blockage or food bridging near the dispensing gate.

Future Recommendations: The following enhancements could further improve the system in future implementations:

IoT Integration: Add an ESP8266 or ESP32 Wi-Fi module to support mobile application notifications and remote feeding control.

Clog Detection: Incorporate a servo current-monitoring circuit or another feedback mechanism to detect mechanical blockage during food dispensing.

Battery Backup: Add a rechargeable battery backup system to maintain system operation during temporary power interruptions.

20. Cost Analysis

Economic viability is an important consideration in the design of practical embedded systems. This section presents the estimated cost of the hardware components used in the proposed Automatic Pet Feeding System and evaluates its overall cost-effectiveness compared with commercially available automatic pet feeders.

20.1 Component Cost Breakdown

The estimated cost of constructing the system prototype is summarized in Table 20.1. The prices represent approximate market values for individual components and may vary depending on supplier, region, and purchase quantity.

Item	Component Description	Model / Specification	Quantity	Unit Cost (USD)	Total Cost (USD)
1	Microcontroller Board	Arduino Uno R3 (ATmega328P)	1	$6.00	$6.00
2	Real-Time Clock Module	DS1307 RTC (with CR2032 battery)	1	$1.50	$1.50
3	Alphanumeric Display	16×2 LCD with I²C Interface	1	$3.00	$3.00
4	Dispensing Actuator	SG90 Micro Servo Motor	1	$2.50	$2.50
5	Distance Sensor	HC-SR04 Ultrasonic Sensor	1	$1.50	$1.50
6	Matrix Keypad	4×4 Membrane Keypad	1	$1.50	$1.50
7	Audio Indicator	Active Piezo Buzzer (5 V)	1	$0.50	$0.50
8	Status LEDs & Resistors	Red LED, Green LED, 220 Ω Resistors	1 Set	$0.50	$0.50
9	Power Supply	Regulated 5 V / 2 A DC Adapter	1	$3.50	$3.50
10	Prototyping Hardware	Breadboard, Jumper Wires, Enclosure	1 Set	$3.00	$3.00
Total	Complete System Cost	—	—	—	$23.50

20.2 Economic Feasibility and Comparative Analysis

Cost Effectiveness: Commercial automatic pet feeders with programmable feeding schedules and low-food warning features typically cost between $50 and $120 USD. With an estimated hardware cost of approximately $23.50 USD, the proposed system provides a cost-effective alternative while offering the essential functions required for automated pet feeding.
Maintenance and Repairability: The system is constructed using standard, widely available electronic modules, making maintenance straightforward and inexpensive. Individual components can be replaced independently without requiring replacement of the entire system, thereby reducing long-term maintenance costs.
Scalability: The modular hardware design supports future product development and large-scale manufacturing. Production costs could be significantly reduced through custom PCB integration, optimized enclosure design, and bulk component purchasing, making the system suitable for commercial production.

20.3 Cost Analysis Summary

The estimated prototype cost demonstrates that the proposed Automatic Pet Feeding System is both technically practical and economically feasible. By combining low-cost off-the-shelf components with reliable embedded control, the system achieves the required functionality while maintaining a significantly lower implementation cost than many commercially available alternatives. This makes the proposed design well suited for educational projects, prototype development, and future commercial enhancement.

21 .References

[1] J. Schmidt, *Arduino: A Technical Reference:* A Handbook for Technicians, Engineers, and Makers, 1st ed. Sebastopol, CA: O'Reilly Media, 2016.

[2] M. Margolis, *Arduino Cookbook: Recipes to Begin, Expand, and Enhance Your Projects*, 3rd ed. Sebastopol, CA: O'Reilly Media, 2020.

[3] S. Monk, *Programming Arduino: Getting Started with Sketches,* 2nd ed. New York: McGraw-Hill Education, 2016.

[4] Microchip Technology Inc., *"ATmega328P 8-bit AVR Microcontroller with 32K Bytes In-System Programmable Flash Datasheet,"* Document DS40002061B, 2018.

[5] Maxim Integrated, *"DS1307 64 x 8, Serial, I2C Real-Time Clock Datasheet,"* Rev. 14, 2008.

[6] Robot Electronics, *"Ultrasonic Ranging Module HC-SR04 Technical Specification,"* Datasheet, 2018.

[7] Tower Pro, *"SG90 9g Mini Servo Motor Datasheet,"* Specification Guide, 2017.

[8] D. Simon, *Embedded Software for System on a Chip*, 1st ed. Indianapolis, IN: New Riders, 1999.

[9] T. Kuphaldt, *Lessons in Electric Circuits*, Volume IV - Digital, 4th ed., 2007. [Online]. Available: http://www.ibiblio.org/kuphaldt/electricCircuits/Digital/

[10] Wokwi, "Wokwi Online ESP32, STM32, Arduino Simulator Documentation," 2026. [Online]. Available: https://docs.wokwi.com/